

ACHEEVY Standalone Platform - Build Foundation Document

Executive Summary

ACHEEVY is a voice-first AI business orchestration platform that serves as the primary user interface and orchestrator for complex workflows. This document provides the foundational specifications for building a standalone ACHEEVY web application on Hostinger infrastructure.

Core Value Proposition: "Think It. Speak It. ACHEEVY Builds It."

ACHEEVY functions as the Digital CEO—a single conversational interface that orchestrates backend AI agents, tools, and services without delegating away from the user experience. It IS the delegation layer itself.

Platform Architecture Overview

Primary Components

- **Frontend Application:** Next.js 14 with App Router architecture
- **Authentication:** Firebase Auth with Google, Telegram, WhatsApp
- **Database:** Firestore with Row-Level Security (RLS)
- **Backend Orchestration:** OpenClaw (containerized as backend_acheevy)
- **Execution Engine:** KiloCode (containerized Python sandbox)
- **Hosting Infrastructure:** Hostinger VPS with Docker containerization
- **Voice Integration:** ElevenLabs Scribe v2 Realtime for speech-to-text
- **AI Routing:** OpenRouter for multi-model LLM access

Design Philosophy

The platform embodies "Apple Glass meets Nothing Tech" aesthetic—premium, minimalist, technology-forward design with emphasis on transparency, clarity, and sophisticated user experience.

Color Palette:

- Primary: #08084 (teal) - Trust and AI empowerment
- Surface: Glass morphism with subtle transparency
- Typography: Clean, modern sans-serif
- Interactions: Smooth animations, voice-aware feedback

Technical Stack Specifications

Frontend Technology

Framework: Next.js 14 with App Router

- Server Components for authentication and data fetching
- Client Components for voice interaction and dynamic UI
- File-based routing via app/ directory structure
- API routes for voice endpoints and agent communication

UI Framework: Shadcn/UI + TailwindCSS v4

- Pre-built accessible components (Button, Input, Slider, Select)
- Utility-first CSS with no inline styles
- Responsive design patterns
- Dark mode support

State Management: React Server Components + Client state

- Server-side data fetching with automatic caching
- Client-side interactivity for voice and real-time features
- Firebase SDK for authentication state

Backend Architecture

Container Orchestration: Docker Compose on Hostinger VPS

Service	Container	Port
OpenClaw	backend_acheevy	8000
KiloCode Executor	kilo_code_executor	8001
Next.js Frontend	frontend_app	3000
Nginx Reverse Proxy	nginx_proxy	80/443

Table 1: Container Service Mapping

Security Isolation:

- All services run in isolated Docker containers
- Non-root execution for all containers
- Read-only filesystem where applicable
- Resource limits (CPU/memory) enforced
- Custom Docker network (locale_net) for inter-service communication
- No direct port exposure—all traffic through Nginx proxy

Database Schema (Firestore)

Collections Structure:

1. **users** - User profiles and authentication metadata
2. **workspaces** - User workspaces with RLS rules
3. **conversations** - Chat history with ACHEEVY
4. **agent_logs** - Execution logs from OpenClaw/KiloCode
5. **luc_usage** - Locale Usage Calculator tracking (token costs)

Row-Level Security: Firestore Security Rules enforce per-user data isolation ensuring users can only access their own workspaces, conversations, and usage data.

Voice-First Interaction System

Voice Pipeline Architecture

Speech-to-Text: ElevenLabs Scribe v2 Realtime

- Real-time streaming transcription
- Cost: \$0.005 per minute (\$5 per 1000 minutes)
- Accuracy optimized for business conversations
- WebSocket connection for low latency

Text-to-Speech: ElevenLabs Voice Generation

- Voice identity: Boomer_Ang (custom cloned voice)
- Natural conversation flow
- Emotional tonality support

Interaction Flow:

1. User speaks into microphone → Scribe v2 transcribes in real-time
2. Transcription sent to ACHEEVY orchestrator
3. ACHEEVY processes intent and delegates to appropriate backend agent
4. Response generated via LLM (routed through OpenRouter)
5. Text-to-speech converts response to audio
6. Audio played back to user while visual feedback displays

AI Orchestration Layer

ACHEEVY as Primary Orchestrator

ACHEEVY does NOT delegate orchestration responsibility—it embodies orchestration. The user never speaks to subordinate agents; they only interact with ACHEEVY, which internally coordinates:

- **OpenClaw** - Multi-agent backend orchestration for heavy-duty workflows
- **KiloCode** - Code execution sandbox for running user scripts
- **Agent Zero** - Conversational AI capabilities (cloned as AVVA NOON)
- **ii-agent** - Core conversational framework
- **II-Commons** - RAG and knowledge base retrieval

LLM Routing Strategy

OpenRouter Integration: Aggregates multiple LLM providers

- Primary models: Claude 3.5, GPT-4, Gemini Pro
- Automatic failover between providers
- Cost optimization via model selection
- Usage tracking for Locale Usage Calculator (LUC)

Model Selection Logic:

- Simple queries → Fast, cost-efficient models (Gemini Flash)
- Complex reasoning → Advanced models (Claude 3.5 Sonnet)
- Code execution → Specialized code models (Deepseek Coder)
- Voice synthesis → ElevenLabs dedicated pipeline

User Experience Design

Authentication Flow

Login Options: Google, Telegram, WhatsApp

- Firebase Authentication SDK
- OAuth 2.0 flows for Google
- Telegram Bot API integration
- WhatsApp Business API integration (future)

Onboarding Animation: Pixel particles animate from ACHEEVY logo into selected authentication button, creating fluid visual continuity.

Core User Interface Components

Dashboard Layout:

- Glass-morphic cards with subtle backdrop blur
- Voice activation button (always accessible)
- Conversation history sidebar
- Real-time LUC cost display
- Agent status indicators

Voice Interaction Widget:

- Persistent floating button in bottom-right corner
- Visual feedback during listening (pulsing animation)
- Real-time transcription display
- Voice activity detection
- Push-to-talk and continuous modes

LUC (Locale Usage Calculator):

- Real-time token usage tracking
- Cost preview before executing AI-intensive actions
- Historical usage charts

- Budget alerts and notifications

Deployment Architecture

Hostinger VPS Configuration

Server Specifications:

- Plan: Cloud Startup VPS (KVM)
- OS: Ubuntu 24.04 LTS
- Node.js: v20 (via nvm)
- Docker: Latest stable version
- Nginx: Reverse proxy configuration
- SSL: Let's Encrypt certificates

Container Deployment Strategy

Docker Compose Structure:

```
version: '3.8'
services:
  frontend:
    build: ./frontend
    container_name: acheevy_frontend
    ports:
      - "3000:3000"
    environment:
      - FIREBASE_CONFIG
      - OPENROUTER_API_KEY
    networks:
      - acheevy_net

  backend_acheevy:
    image: openclaw/openclaw:latest
    container_name: backend_acheevy
    ports:
      - "8000:8000"
    volumes:
      - ./agent_configs:/configs
    networks:
      - acheevy_net

  kilo_executor:
    build: ./kilo-code
    container_name: kilo_code_executor
    ports:
      - "8001:8001"
    security_opt:
      - no-new-privileges:true
    read_only: true
    networks:
      - acheevy_net
```

networks:
acheevy_net:
driver: bridge

Nginx Reverse Proxy Configuration

Traffic Routing:

- / → Frontend Next.js app (port 3000)
- /api/orchestrate → OpenClaw backend (port 8000)
- /api/execute → KiloCode executor (port 8001)
- /api/voice → Voice processing endpoints

SSL/TLS Configuration:

- Automatic HTTPS redirection
- Let's Encrypt certificate auto-renewal
- TLS 1.3 enforcement
- HSTS headers enabled

Cost Optimization Strategy

Infrastructure Costs

Service	Monthly Cost
Hostinger Cloud VPS	\$25.00
Firebase (Spark Plan)	\$0.00
OpenRouter (est. usage)	\$50.00
ElevenLabs Voice API	\$30.00
Domain & SSL	\$5.00
Total	\$110.00

Table 2: Estimated Monthly Operating Costs

Performance Optimization

Frontend:

- Static generation for landing pages
- Incremental static regeneration for dynamic content
- Image optimization via Next.js Image component
- Code splitting and lazy loading
- CDN caching for static assets

Backend:

- Container resource limits prevent runaway processes
- Connection pooling for database queries
- Redis caching layer (future enhancement)

- Request rate limiting on API routes

Security Implementation

Authentication Security

- Firebase Auth with email verification required
- JWT token validation on all API routes
- Session timeout after 30 minutes of inactivity
- Multi-factor authentication support (future)

Data Protection

- Firestore Security Rules enforce strict RLS
- All sensitive data encrypted at rest
- HTTPS-only communication
- API keys stored in environment variables, never in code
- Regular automated backups to Firebase Storage

Container Security

- Non-root user execution in all containers
- Read-only root filesystems where possible
- Minimal base images (Alpine Linux)
- No privileged container access
- Network isolation between containers
- Regular security updates via automated CI/CD

Development Workflow

Local Development Setup

1. Clone repository: `git clone [repo-url]`
2. Install dependencies: `npm install`
3. Configure environment variables in `.env.local`
4. Start Docker services: `docker compose up -d`
5. Run development server: `npm run dev`
6. Access at `http://localhost:3000`

Continuous Integration/Deployment

GitHub Actions Pipeline:

1. Code push triggers automated tests
2. Linting and type checking (TypeScript)
3. Unit tests for critical functions
4. Integration tests for API routes
5. Build Next.js production bundle
6. Build Docker images
7. Push images to container registry
8. Deploy to Hostinger VPS via SSH
9. Run post-deployment health checks

Testing Strategy

Test Coverage Requirements:

- Unit tests: 80% coverage minimum
- Integration tests: All API endpoints
- E2E tests: Critical user journeys (auth, voice interaction, agent execution)
- Performance tests: API response times under 200ms

Monitoring and Observability

Application Monitoring

- Firebase Analytics for user behavior tracking
- Custom logging to Firebase Cloud Logging
- Error tracking via Sentry integration
- Real-time performance metrics dashboard

Container Monitoring

- Docker container health checks
- Resource usage monitoring (CPU, memory, disk)
- Network traffic analysis
- Log aggregation from all containers

Cost Monitoring

LUC System:

- Token usage tracked per user, per conversation
- Real-time cost estimation before action execution
- Monthly usage reports
- Budget alerts when approaching limits
- Historical cost analysis and trends

Roadmap and Milestones

Phase 1: Foundation (Weeks 1-2)

- Set up Hostinger VPS with Ubuntu 24.04
- Configure Docker and Docker Compose
- Deploy Nginx reverse proxy
- Initialize Firebase project
- Create Next.js 14 application skeleton
- Implement basic authentication flow

Phase 2: Core Features (Weeks 3-5)

- Integrate OpenClaw backend orchestration
- Deploy KiloCode execution environment
- Build voice interaction pipeline (ElevenLabs)
- Implement LUC tracking system
- Create user dashboard with conversation history
- Deploy Firestore security rules

Phase 3: Polish and Testing (Weeks 6-7)

- Complete UI/UX design system implementation
- Add animations and micro-interactions
- Comprehensive testing (unit, integration, E2E)
- Performance optimization
- Security audit
- Documentation completion

Phase 4: Launch Preparation (Week 8)

- Final deployment to production VPS
- SSL certificate configuration
- Domain DNS setup
- Load testing and optimization
- User acceptance testing with beta users
- Marketing materials and landing page

Key Success Metrics

Technical Metrics

- API response time: < 200ms (p95)
- Voice transcription latency: < 500ms
- Container uptime: 99.5% availability
- Frontend load time: < 1.5s (First Contentful Paint)
- Code test coverage: > 80%

Business Metrics

- User activation rate: > 60% (complete onboarding)
- Daily active users: Target 100 in first month
- Average session duration: > 5 minutes
- Voice interaction adoption: > 40% of users
- Customer satisfaction (NPS): > 50

Cost Efficiency

- Cost per active user: < \$2/month
- LLM token cost per conversation: < \$0.10
- Infrastructure cost as % of revenue: < 20%

Conclusion

This foundational document provides comprehensive specifications for building ACHEEVY as a standalone platform on Hostinger infrastructure. The architecture balances cutting-edge AI orchestration capabilities with cost-effective deployment, security best practices, and exceptional user experience.

Core Differentiators:

- Voice-first design prioritizing natural conversation
- Single orchestration interface (ACHEEVY) managing complex backend workflows
- Transparent cost tracking via LUC system
- Premium "Apple Glass meets Nothing Tech" aesthetic
- Secure containerized architecture with full isolation
- Scalable foundation supporting future growth

By following this blueprint, development teams can build a production-ready ACHEEVY platform that delivers sophisticated AI orchestration capabilities through an intuitive, voice-driven interface—embodying the vision of "Think It. Speak It. ACHEEVY Builds It."